



# 数据结构

## Data Structure

许 蕾

[xlei@nju.edu.cn](mailto:xlei@nju.edu.cn)



## 第四章 数组、串与广义表



- 一维数组与多维数组
- 特殊矩阵
- 稀疏矩阵
- 字符串
- 广义表



## 4.5 广义表 (General Lists)



- 广义表是  $n$  ( $\geq 0$ ) 个表元素组成的有限序列，记作

$$LS(a_1, a_2, a_3, \dots, a_n)$$

- $LS$  是表名， $a_i$  是表元素，可以是表（称为子表）或数据元素（称为原子）。
- $n$  为表的长度， $n = 0$  的广义表为空表。



- 线性表(Linear Lists)



表头

表尾

$$G = (\underbrace{'a'}_{\text{原子}}, \underbrace{(5, 3, 'x')}_{\text{子表}}, \dots)$$

原子

子表

- $n > 0$ 时，表的第一个表元素称为广义表的表头（head），除此之外，其它表元素组成的表称为广义表的表尾（tail）。



# 广义表的特性



- 有次序性
- 有深度
- 可递归
- 有长度
- 可共享

A( )

A长度为0, 深度为1

B( 6, 2 )

B长度为2, 深度为1

C( 'a', ( 5, 3, 'x' ) )

C长度为2, 深度为2

D( B, C, A )

D长度为3, 深度为3

E( B, D )

E长度为? 深度为?

F( 4, F )

F长度为? 深度为?



# 广义表的表头与表尾



- 广义表的第一个表元素即为该表的表头，除表头元素外其他表元素组成的表即为该表的表尾。

A( )                      head(A) 和 tail(A) 不存在

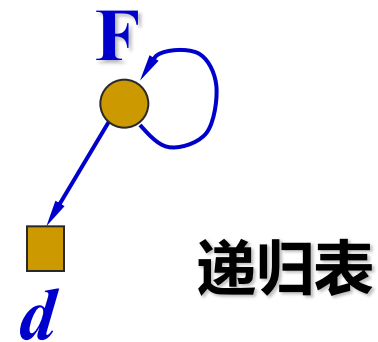
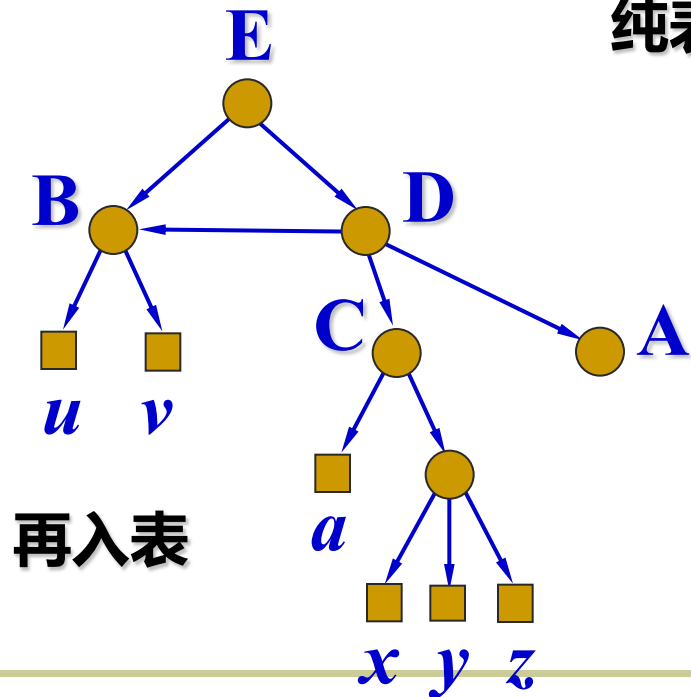
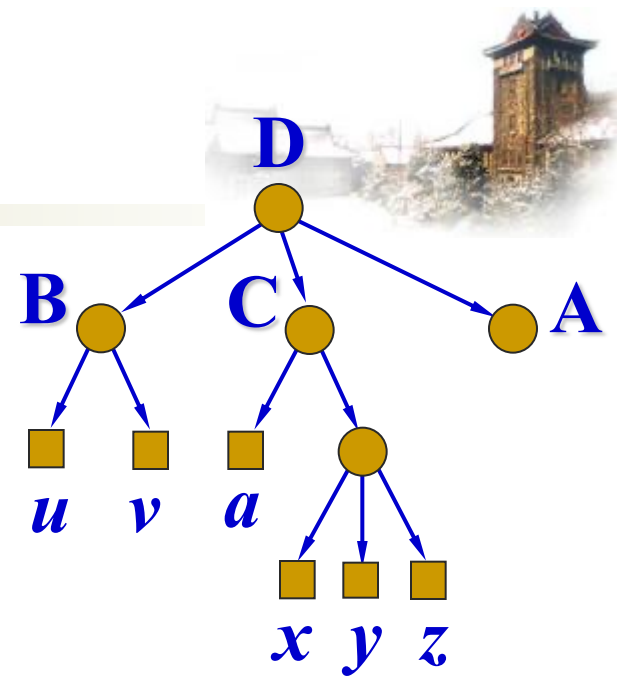
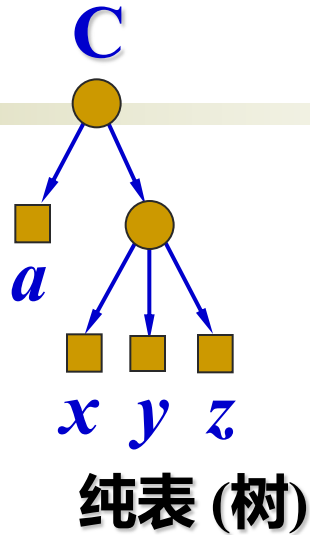
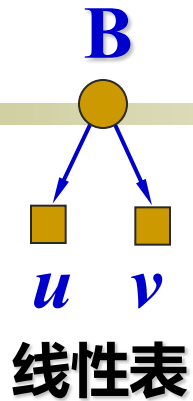
B( 6, 2 )                head(B) = 6, tail(B) = (2)

C( 'a', ( 5, 3, 'x' ) )    head(C) = 'a'    tail(C) = ((5,3, 'x'))

D( B, C, A )                head( ((5,3, 'x')) ) = (5,3, 'x')

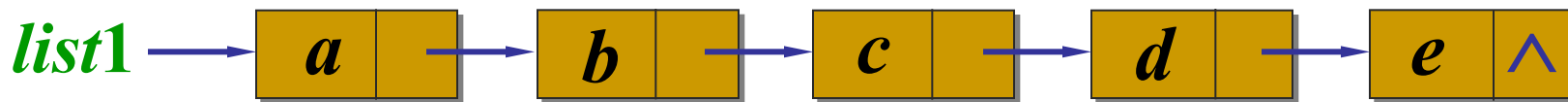
E( B, D )                    tail( ((5,3, 'x')) ) = ( )

F( 4, F )

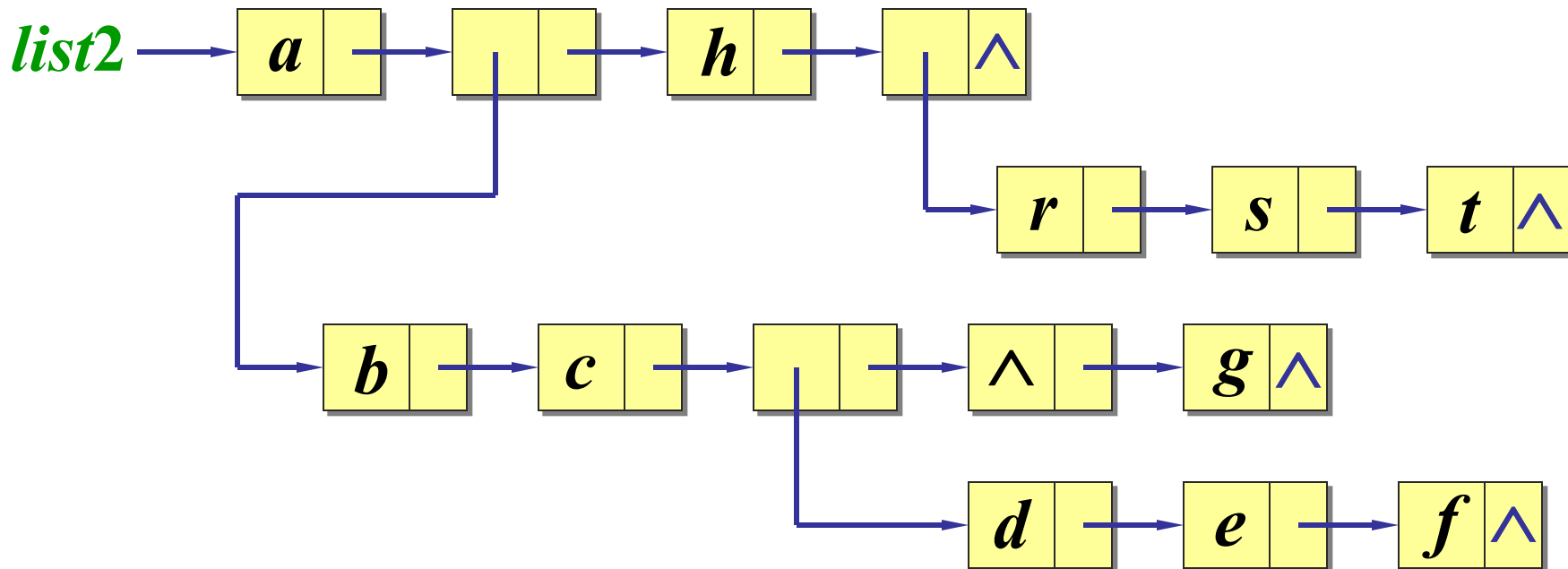




# 广义表的表示



$list1 = (a, b, c, d, e)$



$list2 = (a, (b, c, (d, e, f), \wedge), g, h, (r, s, t))$





# 广义表结点定义

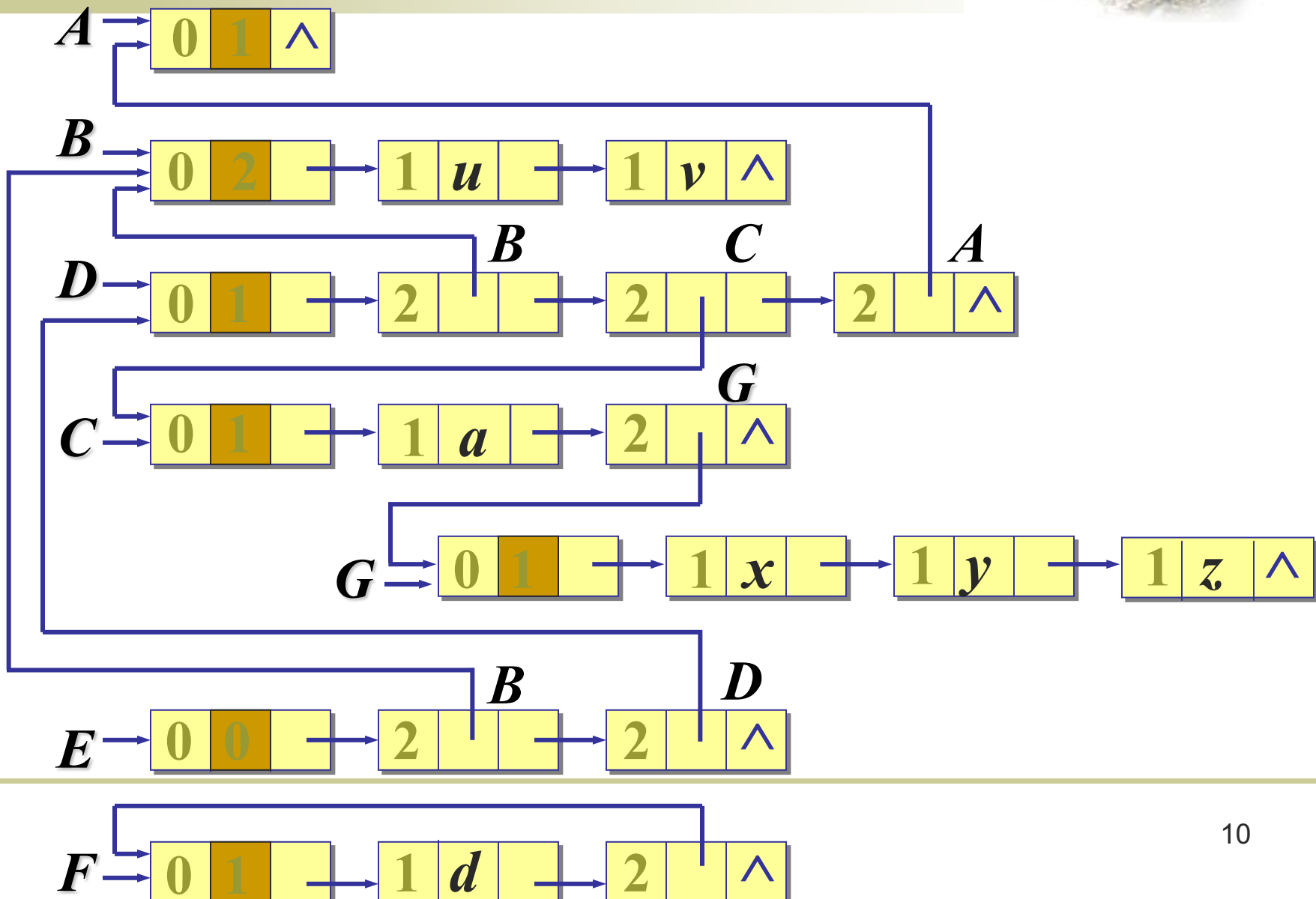


| utype | info | tlink |
|-------|------|-------|
|-------|------|-------|

- **结点类型** utype: = 0, 表头;  
= 1, 原子数据;  
= 2, 子表
- **信息** info: utype = 0时, 存放引用计数(ref);  
utype = 1时, 存放数据值(value);  
utype = 2时, 存放指向子表表头的指针(hlink)
- **尾指针** tlink: utype = 0时, 指向该表第一个结点;  
utype  $\neq$  0时, 指向同一层下一个结点



# 广义表的存储表示





# 广义表的类定义



```
template <class T>
struct GenListNode {                                //广义表结点类定义
    int utype;                                       // = 0 / 1 / 2
    GenListNode<T> *tlink;                          //同层下一结点指针
    union {                                         //等价变量
        int ref;                                   //存放引用计数
        T value;                                   //存放数值
        GenListNode<T> *hlink;                    //存放子表指针
    } info;

    GenListNode()                                  //构造函数
        : utype(0), tlink(NULL), info.ref(0) {}
    GenListNode(GenListNode<T>& R) {
        //复制构造函数

        utype = R.utype;  tlink = R.tlink;
        info = R.info;

    }
};
```



**template <class T>**

**class GenList {**

**//广义表类定义**

**public:**

**Genlist();**

**//构造函数**

**~GenList();**

**//析构函数**

**bool Head (GenListNode<T>& x);** **//x 返回表头元素**

**bool Tail (GenList<T>& lt);** **//lt 返回表尾**

**GenListNode<T> \*First();** **//返回第一个元素地址**

**GenListNode<T> \*Next (GenListNode<T> \*elem);**

**//返回表元素elem的直接后继元素**

**void Copy ( const GenList<T>& R);** **//广义表的复制**

**int Length();** **//计算广义表长度**

**int Depth();** **//计算非递归表深度**



**private:**

```
GenListNode<T> *first; //广义表头指针
GenListNode<T> *Copy (GenListNode<T> *ls);
    //复制一个ls指示的无共享非递归表
int Length (GenListNode<T> *ls);
    //求由ls指示的广义表的长度
int Depth (GenListNode<T> *ls);
    //计算由ls指示的非递归表的深度
bool Equal (GenListNode<T> *s, GenListNode<T> *t);
    //判以s和t为表头的两个表是否相等
void Remove (GenListNode<T> *ls);
    //释放以ls为附加头结点的广义表
```



```
void CreateList (istream &in, GenListNode<T> *& ls,  
SeqList<T>& L1, SeqList <GenListNode<T> *>& L2);  
    //从输入流对象输入广义表的字符串描述,  
    //建立一个带头结点的广义表结构  
  
friend istream& operator >> (istream& in,  
    GenList<T>& L);  
  
};
```



# 广义表类的构造和访问成员函数



```
template <class T>
Genlist<T>::GenList() {           //构造函数
    GenListNode<T> * first = new GenListNode;
    if (first == NULL) { cerr << “存储分配失败! \n”; exit(1); }
};
```

```
template <class T>
bool GenList<T>::Head (GenListNode <T>& x) {
//若广义表非空，则通过x返回其第一个元素的值
//否则函数没有定义
    if (first->tlink == NULL) return false; //空表
    else {                                 //非空表
        x.utype = first->tlink->utype;
        x.info = first->tlink->info;
        return true;                      //x返回表头的值
    }
};
```



```
template <class T>
```

```
bool GenList<T>::Tail(GenList<T>& lt) {
```

```
//若广义表非空，则通过lt返回广义表除表头元素
```

```
//以外其他元素组成的表，否则函数没有定义
```

```
    if (first->tlink == NULL) return false;    //空表
```

```
    else {                                     //非空表
```

```
        lt.first->utype = 0;                   //设置头结点
```

```
        lt.first->info.ref = 0;
```

```
        lt.first->tlink = Copy(first->tlink->tlink);
```

```
        return true;
```

```
    }
```

```
};
```





```
template <class T>
```

```
GenListNode<T> *GenList<T>::First() {
```

```
//返回广义表的第一个元素（若表空，则返回一个特定的  
//空值NULL)
```

```
    if (first->tlink == NULL) return NULL;    //空表
```

```
        else return first->tlink;            //非空表
```

```
};
```



```
template <class T>
GenListNode<T> *GenList<T>::
Next(GenListNode<T> *elem) {
//返回表元素elem的直接后继元素
    if (elem->tlink == NULL) return NULL;
    else return elem->tlink;
};
```



# 广义表的递归算法



- 一个递归算法有两种：
  - 一个是递归函数的外部调用；
  - 另一个是递归函数的内部调用。
- 通常，把外部调用设置为公有函数，把内部调用设置为私有函数。



# 广义表的复制算法



```
template <class T>                                //公有函数
void GenList<T>::Copy(const GenList<T>& R)
    first = Copy(R.first);                          //调用私有函数
};
```



```
template <class T>                //私有函数
GenListNode<T>* GenList<T>::Copy(GenListNode <T> *ls) {
    //复制一个 ls 指示的无共享子表的非递归表
    GenListNode<T> *q = NULL;
    if (ls != NULL) {
        q = new GenListNode<T>;           //处理当前的结点q
        q->utype = ls->utype;              //复制结点类型
        switch (ls->utype) {               //根据utype传送信息
            case 0: q->info.ref = ls->info.ref; break;
            case 1: q->info.value = ls->info.value; break;
            case 2: q->info.hlink = Copy(ls->info.hlink); break;
        }
        q->tlink = Copy(ls->tlink); //处理同层下一结点
    }
    return q;
};
```



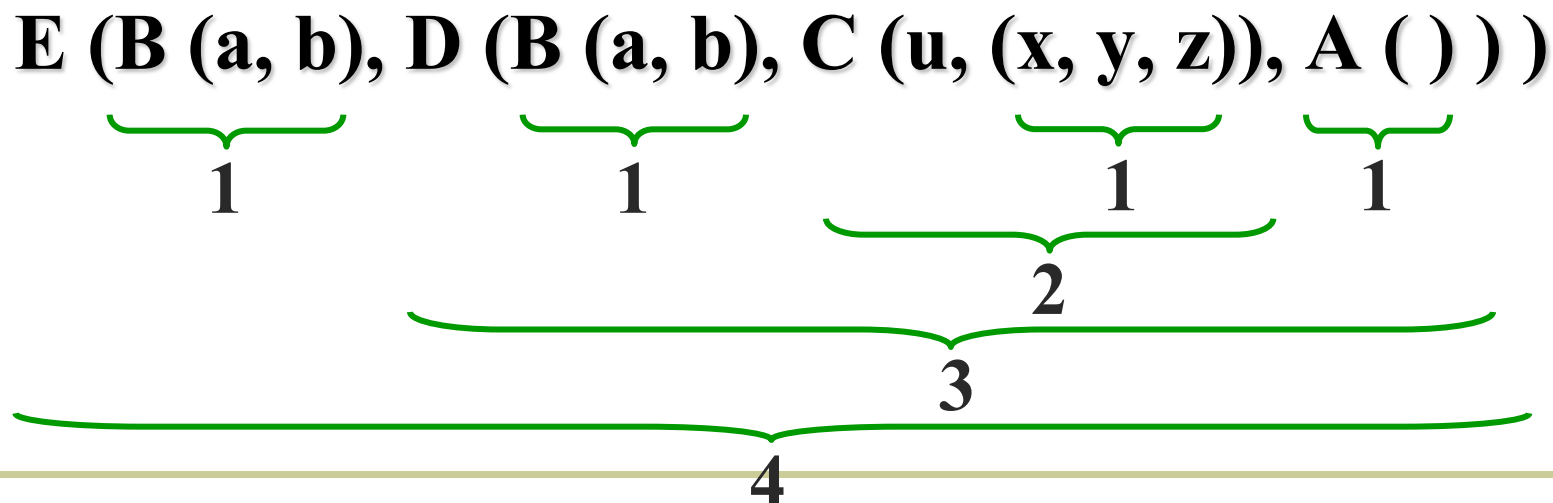


# 求广义表深度的算法



$$\text{Depth}(\text{LS}) = \begin{cases} 1, & \text{当LS为空表时} \\ 0, & \text{当LS为原子时} \\ 1 + \max_{0 \leq i \leq n-1} \{\text{Depth}(\alpha_i)\}, & \text{其他, } n \geq 1 \end{cases}$$

- 例如，对于广义表





```
template <class T>
int GenList<T>::Depth() {           //公有函数
//计算一个非递归表的深度
    return Depth(first);
};
```

```
template <class T>                 //私有函数
int GenList<T>::Depth(GenListNode<T> *ls) {
    if (ls->tlink == NULL) return 1;
        // ls->tlink ==NULL, 空表, 深度为1
    GenListNode<T> *temp = ls->tlink;
    int m = 0, n;
```

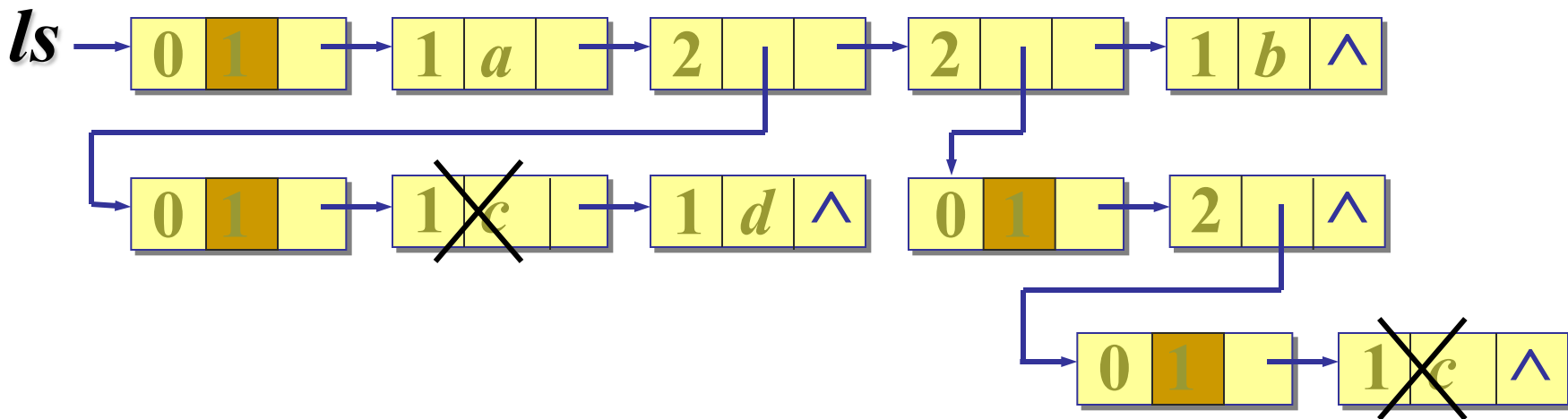




```
while (temp != NULL) { //在广义表顶层横扫
    if (temp->utype == 2) { //扫描到表结点
        n = Depth(temp->info.hlink);
                                //递归计算子表深度
        if (m < n) m = n;      //取最大深度
    }
    temp = temp->tlink;
}
return m+1;                //返回深度
};
```



# 广义表的删除算法



## 扫描子链表

- ◆ 若结点数据为 $c$ , 删除。可能做循环连续删除。
- ◆ 若结点数据不为 $c$ , 不执行删除。
- ◆ 若结点为子表, 递归在子表执行删除。



```
template <class T>
```

```
void delvalue(GenListNode<T> *ls, T x) {
```

```
    if (ls->tlink != NULL) { //非空表
```

```
        GenListNode<T> * p = ls->tlink; //第一个结点
```

```
        while (p != NULL && (p->utype == 1 &&  
            p->info.value == x)) {
```

```
            ls->tlink = p->tlink; delete p;
```

```
            p = ls->tlink; //p指向同层下一结点
```

```
        }
```

```
    if (p != NULL) {
```

```
        if (p->utype == 2) //递归在子表中删除
```

```
            delvalue(p->info.hlink, x);
```

```
            delvalue(p, x); //在以p为表头的链表中递归删除
```

```
        }
```

```
    }
```

```
};
```



## ■ 删除子表:

- 对于共享表来说, 如果一个表元素有多个地方使用它, 贸然删去它, 会造成其他地方使用出错。
- 因此, 当要做删除时, 先把该表的头结点中的引用计数 `ref` 减1, 当引用计数减到 0 时才能执行结点的真正释放。

```
template <class T>
```

```
GenList<T>::~~GenList() {
```

```
//广义表的析构函数, 每个头结点都有引用计数
```

```
    Remove(first);
```

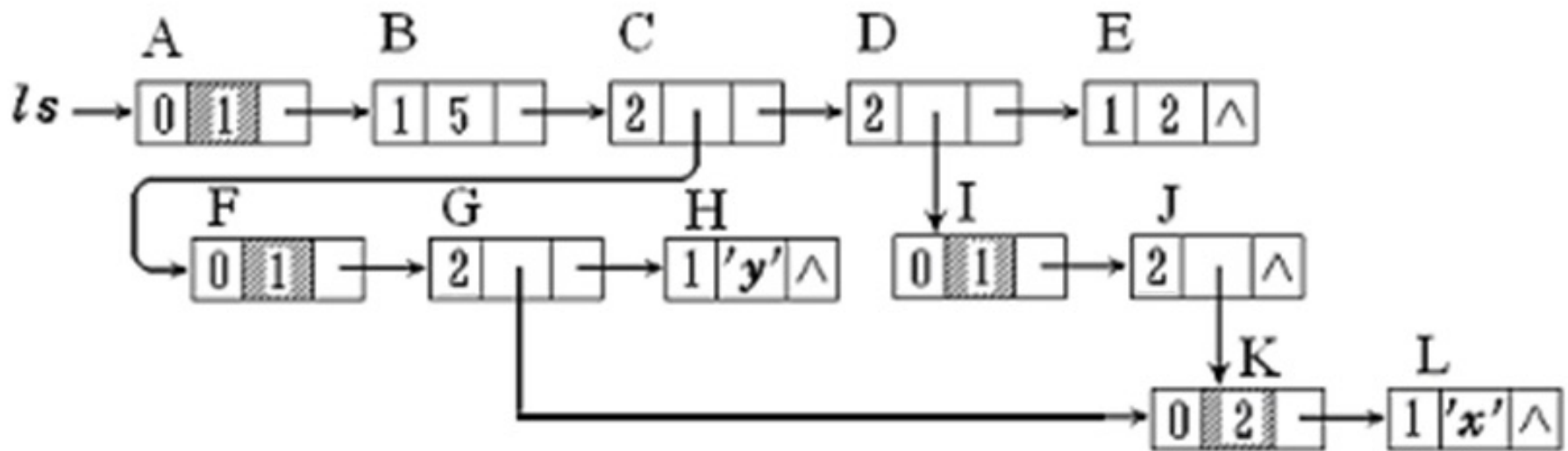
```
};
```



```
template <class T>
void GenList<T>::Remove(GenListNode<T> *ls) {
//私有函数：释放以ls为表头指针的广义表
    ls->info.ref--;           //头结点的引用计数减1
    if (ls->info.ref <= 0) {   //如果减到0
        GenListNode<T> *q;
        while (ls->tlink != NULL) { //横扫表顶层
            q = ls->tlink;           //到第一个结点
            if (q->utype == 2) {      //递归删除子表
                Remove(q->info.hlink);
                if (q->info.hlink->info.ref <= 0)
                    delete q->info.hlink; //删子表头结点
            }
            ls->tlink = q->tlink; delete q;
        }
    }
};
```



- 例：给出下面的广义表进行删除时的结点删除的顺序





# 建立广义表的链表表示



设广义表的字符串描述如下

$S(A( 'b', 'c' ), B( '#' ), 'd' );$

根据字符串描述生成广义表链表结构



- 假设一个广义表的元素类型  $T$  是字符类型 `char`，每一个原子元素的值为英文字母，并假定广义表从输入流对象 `istream` 输入。
- 在算法的执行过程中，检测从输入流对象输入的一个字符，如果遇到表名（用大写字母表示）
  - 检查这个表名是否已经存在
    - 如果是，说明该表是共享表，只要将相应头结点的引用计数加一即可；
    - 如果不是，保存该表名并建立相应广义表。
- 表名后面一定是左括号 `'(`
  - 不是，则输入错；
  - 是，则递归建立广义表结构。





- 如果遇到用小写字母表示的原子，则建立原子结点；
- 如果遇到右括号')'，子表链收尾并退出递归。
- 注意在空表情形括号里应夹入一个非英文字母，如字符'#'，不能一个字符也没有。整个广义表描述字符串以';'结束。



# 由字符串建立广义表的链表表示算法



```
#include <string.h>
template <class T>
void Genlist<T>::CreateList(istream &in,
    GenListNode<T> *& ls, SeqList<T>& L1,
    SeqList <GenListNode<T> *>& L2) {
//从广义表的字符串描述 s 出发, 建立一个带头结点
//的广义表, 要求T为char类型。在表L1存储大写字母
//的表名, 在表L2存储表名对应子表结点的地址。
T chr;
    in >> chr;
```



//读入一个字符，只可能读入#、左括号和字母

```
if (isalpha(chr) && isupper(chr) || chr == '(') {  
    //大写字母或 '('
```

```
    ls = new GenListNode<T>;    //建子表结点
```

```
    ls->utype = 2;
```

```
    if (isalpha(chr) && isupper(chr)) { //表名处理
```

```
        int n = L1.Length();
```

```
        int m = L1.Search(chr);
```

```
        if (m != 0) { //该表已建立
```

```
            GenListNode<T> *p = L2.Locate(m); //查子表地址
```

```
            ls->info.hlink = p;
```

```
            p->ref++; //引用计数加一
```

```
            CreateList(in, ls->tlink, L1, L2);
```

```
            //递归建后继表
```

```
}
```



```
else {    //该表未建立
    L1.Insert(n, chr); L2.Insert(n, ls);
                                //保存表名及地址
    in >> chr;
    if (chr != '(') exit(1);    //表名后必跟'('
    ls->info.hlink = new GenListNode<T>;
    ls->info.hlink->utype = 0;    //建头结点
    ls->info.hlink->ref = 1;
    CreateList(in, ls->info.hlink->tlink, L1, L2);
                                //递归建子表
}
```



37



- 该算法需要扫描输入广义表中的所有字符，且处理每个字符都是简单比较或赋值操作，其时间复杂度为 $O(1)$ ，所以整个算法的时间复杂度为 $O(n)$ ， $n$ 是广义表中所有字符数。
- 算法中既包含向子表的递归调用，也包含向后继表的递归调用，所以递归调用的最大深度不会超过生成的广义表中所有结点数，其空间复杂度也为 $O(n)$ 。



```
template <class T>
istream& operator >> (istream& in, GenList<T>& L)
{
    int n;
    cout << “输入广义表串的字符个数: ” << endl;
    in >> n;
    SeqList<T> Ls1(n);
    SeqList <GenListNode<T> *> Ls2 (n);
    CreateList (in, L.first, Ls1, Ls2);    //建立存储结构
};
```



## 1. 表示数学表达式和公式

这是最经典和直观的应用。广义表可以完美地表示运算符和操作数之间的层次关系。

- 例子：算术表达式  $(a * b) + (c / (d - e))$

我们可以用广义表表示为： $(+, (*, a, b), (/ , c, (-, d, e)))$

- 这里， $+$ ， $*$ ， $/$ ， $-$  是运算符（原子）。
- $a$ ， $b$ ， $c$ ， $d$ ， $e$  是操作数（原子）。
- $(*, a, b)$  和  $(/, c, (-, d, e))$  是子表。

- 实际应用场景：

- 符号计算系统：如 Mathematica、Maple 等，在内部表示复杂的数学公式时，使用的就是一种类似广义表的结构。
- 编译器前端：编译器将源代码解析后，会生成一种称为抽象语法树 的中间表示。AST 本质上就是一种广义表结构，它清晰地表达了程序的语法结构。

为什么好用？ 因为遍历这个广义表，很容易实现表达式的求值、化简、微分等操作。





## 2. 人工智能与专家系统

在AI的早期，Lisp 语言是人工智能研究的主流语言，而 Lisp 语言的程序和数据都统一用广义表（在Lisp中称为S-表达式）来表示。

- 例子：表示知识

一条知识：“如果某动物是哺乳动物且吃肉，那么它是食肉动物。”

可以用广义表表示为：( IF ( AND ( 是 动物 哺乳动物 ) ( 吃 动物 肉 ) ) ( 是 动物 食肉动物 ) )

- 实际应用场景：

- 知识表示：在专家系统中，用于表示产生式规则、事实等。
- 自然语言处理：用于表示句子的语法结构。
- 机器学习：用于表示遗传编程中的程序树。

为什么好用？程序的统一表示使得“元编程”（编写能操作程序的程序）变得非常容易，这是AI研究中的一个强大工具。



### 3. 多层次、异构的数据管理

当数据本身具有不规则的层次结构时，二维表格（如数据库表）就很难表示，而广义表则游刃有余。

- 例子：企业和部门结构

text

复制 下载

```
公司 = (  
  "ABC公司",  
  (  
    "研发部", ("张三", "李四"),  
    "销售部", ("王五", ("华东区", ("赵六", "孙七")), "刘八")  
  )  
)
```

这个广义表清晰地表示了公司、部门、区域和员工之间的多层次从属关系。

- 实际应用场景：

- **配置文件**：复杂的、嵌套的配置文件（如JSON、XML、YAML）在内存中解析后，本质上就是一种广义表结构。例如，一个JSON对象包含数组，数组里又包含对象。
- **NoSQL数据库**：文档型数据库（如MongoDB）存储的文档，其结构就是层次化的，类似于广义表。

为什么好用？ 它能自然地模拟现实世界中对象的聚合关系，比固定的表结构更灵活。





## 4. 实现其他复杂数据结构

广义表由于其递归和共享的特性，可以用来作为实现更复杂数据结构的基础。

- 例子：表示带共享子表达式的图结构

考虑一个表达式：  $(a + b) * (a + b) / c$

我们可以设计一个广义表，让两个  $(a + b)$  指向同一个子表，从而实现共享，节省空间。

表示为：  $( /, ( *, P, P ), c )$ ，其中  $P = ( +, a, b )$

- 实际应用场景：

- 图形学中的场景图：一个复杂的3D场景由多个对象组成，这些对象可能共享相同的子组件（如一颗树模型被多次使用）。场景图可以用广义表来表示。
- 有向无环图：通过共享子表节点，广义表可以表示DAG。

为什么好用？ 它内置了“引用”或“指针”的概念，允许数据共享。



## 第4章书面作业



提交时间：10月9日，上课前提交

- (1) 第183页，4.1
- (2) 第184页，4.9
- (3) 第184页，4.10
- (4) 第184页，4.11
- (5) 计算串a b a b a a b a b a b a的next数组